Defence Technology 29 (2023) 80–94

Contents lists available at ScienceDirect

Defence Technology

journal homepage: www.keaipublishing.com/en/journals/defence-technology

Cooperative multi-target hunting by unmanned surface vehicles based on multi-agent reinforcement learning

Jiawei Xia [a], Yasong Luo [a, *], Zhikun Liu [a], Yalun Zhang [b], Haoran Shi [a], Zhong Liu [a]

[a] College of Weaponry Engineering, Naval University of Engineering, Wuhan, 430033, China
[b] Institute of Vibration and Noise, Naval University of Engineering, Wuhan, 430033, China

ARTICLE INFO

Article history:
Received 25 June 2022
Received in revised form
29 August 2022
Accepted 26 September 2022
Available online 11 October 2022

Keywords:
Unmanned surface vehicles
Multi-agent deep reinforcement learning
Cooperative hunting
Feature embedding
Proximal policy optimization

ABSTRACT

To solve the problem of multi-target hunting by an unmanned surface vehicle (USV) fleet, a hunting algorithm based on multi-agent reinforcement learning is proposed. Firstly, the hunting environment and kinematic model without boundary constraints are built, and the criteria for successful target capture are given. Then, the cooperative hunting problem of a USV fleet is modeled as a decentralized partially observable Markov decision process (Dec-POMDP), and a distributed partially observable multi-target hunting Proximal Policy Optimization (DPOMH-PPO) algorithm applicable to USVs is proposed. In addition, an observation model, a reward function and the action space applicable to multi-target hunting tasks are designed. To deal with the dynamic change of observational feature dimension input by partially observable systems, a feature embedding block is proposed. By combining the two feature compression methods of column-wise max pooling (CMP) and column-wise average-pooling (CAP), observational feature encoding is established. Finally, the centralized training and decentralized execution framework is adopted to complete the training of hunting strategy. Each USV in the fleet shares the same policy and perform actions independently. Simulation experiments have verified the effectiveness of the DPOMH-PPO algorithm in the test scenarios with different numbers of USVs. Moreover, the advantages of the proposed model are comprehensively analyzed from the aspects of algorithm performance, migration effect in task scenarios and self-organization capability after being damaged, the potential deployment and application of DPOMH-PPO in the real environment is verified.

1. Introduction

With the rapid development of unmanned vehicles and intelligent technology, the intelligent combat of unmanned clusters will become a promising battle mode on the future battlefield [1,2]. Compared with Unmanned Aerial Vehicles (UAVs), Unmanned Ground Vehicles (UGVs) and other technically mature equipment, the application of USV fleets in autonomous operations is still in the exploratory stage [3–5]. In this field, multi-target hunting by a USV fleet is an important aspect of autonomous operations of USVs, involving situation awareness, cooperation, decision-making and control, and rivalry game.

To solve the problem of multi-target hunting by a USV fleet, traditionally, the solution methods mainly include the deterministic approach and the heuristic approach. The former solves the problem using traditional mathematical tools. By building a differential game model of pursuer-evader, the optimal time trajectory and hunting strategy can be calculated [6]. Inspired by the hunting behavior in nature, heuristic algorithms simulating wolf hunting [7], predator-prey systems [8] and others have also been proposed. However, in the hunting strategy design, the traditional methods have some limitations as they often make a single assumption about the motion strategy of the escaping target or evader, but it is difficult for one's own side to know the control strategy of the evader in the real battlefield environment, and when the environment model changes, the controller parameters are difficult to adapt quickly.

With the rapid development of Deep Reinforcement Learning (DRL), artificial intelligence has displayed overwhelming advantages in dealing with complex decision-making and game problems [9], and even outperforms human beings in some fields [10,11].

* Corresponding author.
E-mail address: yours_baggio@sina.com (Y. Luo).
Peer review under responsibility of China Ordnance Society

https://doi.org/10.1016/j.dt.2022.09.014

Multi-agent Reinforcement Learning (MARL) is a combination of the Multi-agent System (MAS) and DRL. Under given rules and environmental constraints, through extensive training, agents can show extraordinary intelligence and insight [12,13]. With such characteristics, they can give play to the decision-making advantages in the military environment featuring high-intensity game confrontation. The DRL-based unmanned equipment coordination [14] and air combat training system [15] have been preliminarily applied in the U.S. military.

MARL techniques have seen wide applications in target encirclement and capture. For example, Fu et al. [16] proposed the DE-MADDPG (decomposed multi-agent deep deterministic policy gradient) algorithm to address the problem of cooperative pursuit with multiple UAVs. Four drones were used to capture the high-speed evaders in the simulation environment. Wang et al. [17] developed a learning-based ring communication network to solve the communication optimization problem in the cooperative pursuit process, which could reduce the communication overhead and meanwhile ensure a high task success rate. Wan et al. [18] introduced a novel adversarial attack trick into agent training by modeling uncertainties of the real world, so as to strengthen training robustness. In terms of the kinematics of non-holonomic constraints, Souza et al. [19] realized the cooperative pursuit of agents using the TD3 algorithm [20], and verified the proposed method by drones in the real world. Hüttenrauch et al. [21] studied the scalability of the homogeneous agents model and developed a state representation method that meets the permutation invariance. The experiments showed that this method could support up to 50 agents to participate in the pursuit-evasion task simultaneously.

However, the majority of references focused on the situation of multiple agents capturing a single target, while the research on cooperative multi-target capture is not sufficient. It is largely due to the increased complexity of the problem where compared to single-target capturing, to capture all targets as soon as possible, the agents need to not only learn to cooperate but also approach appropriate targets according to situational awareness. Additionally, during the multi-target capturing task, an agent needs to cooperate with other nonspecific agents multiple times. In order to reduce the complexity of the problem, Ma et al. [22] utilized task assignment and converted multi-target capture into multiple single-target captures. However, as the evader has mobility in the real world, and the situations of the pursuer and the evader are dynamically evolving in the game environment, the task allocation based on the current state is not necessarily optimal. Yu et al. [23] proposed the distributed multi-agent dueling-DQN algorithm to solve the multi-agent pursuit problem. The encirclement of two intruders by eight pursuers in the MAgent environment [24] was realized, proving that it is technically feasible to use the end-to-end training method to enable pursuers to learn evader selection and cooperative pursuit at the same time. However, since the MAgent environment is based on discrete grid spaces, it ignores the kinematic features of agents, and can only represent the abstract spatial relationship between agents, which cannot be migrated to the real environment.

The existing research findings are largely based on the assumptions of finite boundary constraints and global situational awareness, and thus are difficult to be directly applied to the engineering practice of multi-USV hunting. Firstly, most studies have introduced the hunting boundary constraint [16–19,22,23]. When the evader is chased to the regional boundary, it is regarded as successful hunting. Some studies used the periodic toroidal state-space boundary constraint [21]. When the agent moves beyond the boundary, it will reappear from the opposite side of the space, which is obviously impossible in the real scenario. Moreover, as there are usually no boundary constraints and maneuvering range restrictions in the marine environment, the USVs cannot drive the target to the edge of the area, but only surround its by themselves, and thus it is more difficult to achieve successful hunting. Secondly, some studies assume that the environment is fully observable [16–18,22]. However, in the real battlefield environment, a large number of agents distributed widely in space are involved in the cooperative task. Affected by data processing capabilities and communication distance, each agent is usually unable to obtain the global situation information in the environment, which brings the problem of partial observability. In addition, unmanned platforms in the confrontational environment are required to have the ability to continue to perform tasks when a small number of nodes are damaged. However, the impact of node damage on hunting tasks has not been sufficiently investigated.

Driven by the problems above, a hunting environment model without boundary constraints is built and the criteria for successful USV hunting are given. On this basis, the USV motion model and the evasive maneuver policy of the evader are developed. Then, the multi-USV cooperative hunting problem is modeled as a decentralized partially observable Markov decision process (Dec-POMDP) [25], and a distributed partially observable multi-target hunting Proximal Policy Optimization (DPOMH-PPO) algorithm applicable to USVs is proposed. The observation model based on partial observable conditions is designed, and the information to assist agents in making round-up and capture decisions is extracted. A reward function combining guiding rewards and sparse rewards is constructed, covering the rewards for agents in the whole process from target selection to capture. The action space decoupled from kinematic control of USVs is designed to avoid the involvement of the policy network in the complex coupling relationship between USV kinematic control and the reward function. A policy network and a value network based on feature embedding block are also designed to give the algorithm more flexibility and robustness. Finally, the simulation environment of multi-USV hunting is built, the network training of DPOMH-PPO is completed by using curriculum learning, and the effectiveness of the algorithm is analyzed and compared with other algorithms in terms of performance.

Salient contributions in this paper can be summarized by three aspects:

(1) Proposing a multi-USV cooperative hunting method for multiple targets by using the MARL method to solve problems with continuous, no boundary limits and partial observability in practical applications.
(2) Using the end-to-end training method, designing the state feature and reward function for the whole task process, and realizing the agents' decision-making processes such as objective allocation and cooperative hunting through a single policy network.
(3) Introducing the feature embedding block applicable to agents under partially observable conditions, which enables the policy network to deal with the observational features of dynamic dimensionality efficiently, thus making the USV fleet more flexible and robust.

The rest of this paper is structured as follows. Section 2 introduces the preliminaries related to the MARL model and algorithm. Section 3 describes the multi-target hunting problem and establishes the kinematic model. Section 4 covers the design details of the DPOMH-PPO algorithm in detail, including multi-agent state

space, reward function and action space, network structure and algorithm flow, etc. Section 5 gives the configuration of environment and training parameters, and analyzes the DPOMH-PPO algorithm from several angles. In Section 6, the main conclusions are presented.

2. Preliminaries

2.1. Local observation models

The interaction graph denoted by $\mathscr{G} = (V, E)$ is determined by the set of nodes $V = \{v_1, v_2, \ldots, v_N\}$ and the set of edges E of the agent swarm. $E \subset V \times V$ is represented by the undirected edges in the form of $\{v_i, v_j\}$, where agents i and j are neighbors. $\mathscr{G}$ is called a dynamic interaction graph if the set of nodes or the set of edges undergo changes. The neighbors of agent i in the graph $\mathscr{G}$ is given by

$$\mathscr{N}_{\mathscr{G}}(i) = \{j | \{v_i, v_j\} \in E\} \tag{1}$$

Within this neighborhood, agent i can sense local information about other agents. The information agent i receives from agent j is denoted as $o_{i,j} = f(s_i, s_j)$, It is a function of the states of agent i for agent j. Only when $j \in \mathscr{N}_{\mathscr{G}}(i)$, observation $o_{i,j}$ is available for agent i. Hence, the complete information set agent i receives from all neighbors is given by $O_i = \{o_{i,j} | j \in \mathscr{N}_{\mathscr{G}}(i)\}$.

2.2. Decentralized partially observable Markov decision model (Dec-POMDP)

The decentralized partially observable Markov decision process (Dec-POMDP) [25] with shared rewards is investigated. A Dec-POMDP is constructed as $\langle \mathscr{S}, \mathscr{A}, \mathscr{O}, O, \mathscr{P}, R, N \rangle$, where $\mathscr{S}$ is the state space, $\mathscr{A}$ is the shared action space for each agent, and $\mathscr{O}$ is the set of local observations. $O : \mathscr{S}^N \times \{1, \ldots, N\} \to \mathscr{O}$ denotes the observation model, which determines the local observation $o_i \in \mathscr{O}$ for agent i at a given state $\boldsymbol{s} \in \mathscr{S}^N$, namely $o_i = O(\boldsymbol{s}, i)$, and $\mathscr{P} : \mathscr{S}^N \times \mathscr{A}^N \times \mathscr{S}^N \to [0, 1]$ denotes the transition probability model from $\boldsymbol{s}$ to $\boldsymbol{s}'$, given the joint action $\boldsymbol{a} \in \mathscr{A}^N$ for all N agents. Finally, $R : S^N \times A^N \to \mathbf{R}$ represents the global reward function for encoding the cooperative task of the agents by providing an instantaneous reward feedback $R(\boldsymbol{s}, \boldsymbol{a})$ according to the current state $\boldsymbol{s}$ and the corresponding joint action $\boldsymbol{a}$ of the agents.

2.3. PPO algorithm

Proximal Policy Optimization [26] (PPO) is a reinforcement learning method based on on-policy and actor-critic framework. Originating from Trust Region Policy Optimization (TRPO) algorithm [27], this algorithm reduces the computational complexity and improves the learning performance by modifying the objective function. PPO algorithm takes the probability ratio of the output actions of strategies before and after policy update as its basis. The policy gradient loss function is defined as

$$L^{CLIP}(\theta) = \widehat{\boldsymbol{E}}_t[\min(r_t(\theta)\widehat{A}_t, \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\widehat{A}_t)] \tag{2}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$ represents the importance sampling weight, ε is a hyperparameter, the clip function is to limit $r_t(\theta)$ within the interval $[1-\varepsilon, 1+\varepsilon]$, and $\widehat{A}_t$ denotes the estimation of the advantage function [28], whose expression is

$$\widehat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \ldots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \tag{3}$$

3. Problem modeling

3.1. Description of multi-target hunting by USV fleet

Let's consider the following scenarios. There are N_p homogeneous USVs and N_p evaders in the boundless sea. The two sides have opposite tactical purposes. The USVs need to cooperate to capture the evaders, and the evaders try to avoid and stay away from the USVs. When an evader is successfully encircled by the USVs, it will be neutralized, and then the USVs can choose to continue to hunt other targets that have not been neutralized. When all N_p evaders are neutralized, the task is completed.

The scenario of single-target hunting is shown in Fig. 1.

In Fig. 1, $T_m (m = 1, 2, \ldots, N_e)$ represents for any evader, d_{cap} is the hunting radius, $U_i (i = 1, 2, \ldots, N_p)$ represents the USVs, and $d_{i,k}$ is the distance between U_i and T_m. When the distance between the USV and T_m is less than the encirclement radius d_{cap}, the USV is seen as being in the process of pursuing T_m. The encirclement angle $\alpha_{i,j}$ is defined as the included angle of U_i and U_j relative to T_m when encircling the escaping target T_m. If there are no other USVs between the azimuth angle of two USVs relative to T_m, the relationship between the two USVs is defined as neighboring. If among the USVs participating in the pursuit, the encirclement angle between any neighboring USVs is less than α_{cap}, then the T_m is successfully captured. α_{cap} is the valid encirclement angle.

Limited by the observation and communication distance, each USV can only obtain the state of the surrounding friendly USVs and the location information of the evaders. It is necessary to establish an effective strategy so that each USV can make maneuver decisions based on the received information, and that the USV fleet can successfully hunt multiple targets in the shortest time possible without collisions between them.

3.2. Kinematic model

For both sides of the pursuer and the evader, the second-order kinematic equation is established as follows:

$$\begin{cases} \dot{v}_i = a_v \\ \dot{\omega}_i = a_\omega \\ \dot{x}_i = v_i \cos\psi_i \\ \dot{y}_i = v_i \sin\psi_i \\ \dot{\psi}_i = \omega_i \end{cases} \tag{4}$$

where (x_i, y_i) represents the position of any agent i; v_i, ψ_i, ω_i are the

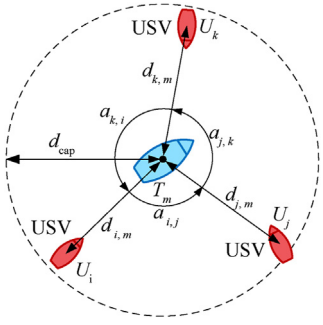

Fig. 1. Schematic diagram of single-target hunting.

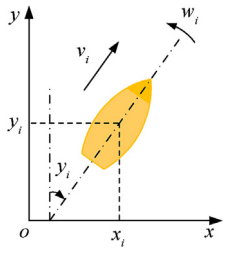

Fig. 2. Schematic of the agent coordinate system.

speed, heading angle and angular velocity respectively; a_v and a_ω are acceleration and angular acceleration. Among them, speed v_i and angular velocity ω_i are restricted: $-\omega_{\max} \le \omega_i \le \omega_{\max}, 0 \le v_i \le v_{\max}$. Fig. 2 shows the coordinate system of the agent.

3.3. Strategy of the evader

This paper employs the artificial potential field method, which is commonly used in path planning, as the escape strategy of the evader. It is assumed that each USV applies a repulsive force in the vector direction of the evader, and each repulsive force component decreases with the increasing distance between the USV and the target. The velocity direction vector $\boldsymbol{v}_m$ of the evader T_m is expressed as

$$\boldsymbol{v}_m = \sum_i \left(\frac{\boldsymbol{p}_i - \boldsymbol{p}_m}{d_{i,m}^2} \right) \tag{5}$$

where $\boldsymbol{p}_i(x_i, y_i)$ and $\boldsymbol{p}_m(x_m, y_m)$ represent the positions of the USV i and the evader T_m respectively.

4. DPOMH-PPO algorithm design

The multi-target cooperative hunting task of the USV fleet can be described as a decentralized partially observable Markov decision model (Dec-POMDP), which is represented by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, O, \mathcal{P}, R, N \rangle$.

The model has two important characteristics: 1) All agents can only observe part of the environment state $o^i = O(\boldsymbol{s}, i)$ via the observation model O; 2) All homogeneous agents share the same policy π_θ, and θ is the parameterized policy network weight. To ensure the homogeneity of the system, it is considered that the state transition model $\mathcal{P}$ and the observation model O of the agents are permutation invariant [29].

The polity π of agent i is the mapping probability from local observation o^i to action a^i. At any time t, the state of the system is $\boldsymbol{s}_t \in \mathcal{S}^N$, agent i obtains the local state $o_t^i = O(\boldsymbol{s}_t, i)$ according to the observation model O, and selects the action $a_t^i \in \mathcal{A}$ through the policy π. The system reaches the next state $\boldsymbol{s}_{t+1} \in \mathcal{S}^N$ through the state transition model $\mathcal{P}$, and each agent receives the same reward $R(\boldsymbol{s}, \boldsymbol{a})$. The purpose of the reinforcement learning algorithm is to find the optimal policy π^* so as to maximize the cumulative discounted reward R_t.

$$\begin{aligned} R_t &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots \\ &= \sum_{k=0}^{\infty} \gamma^k r_{t+k} \end{aligned} \tag{6}$$

where γ is a parameter, $0 \le \gamma \le 1$, called the discount rate.

In this section, combined with learning in an end-end manner, such components of Dec-POMDP as the observation model, reward function and action space used in the DPOMH-PPO algorithm were first designed, and then the DPOMH-PPO algorithm flow was studied. Finally, the network design was introduced in detail.

4.1. Observation model design

The system state is a complete description of a hunting scenario. Due to the partial observability of the system, an agent cannot obtain all the states and is only able to observe limited information. Thus, it is necessary to establish the observation model O to convert the partially observable system state into decision information that can be directly used by the agent.

The system state in the hunting problem consists of the position of the USV fleet, motion parameters, the position of the evader and the capture state. And there are two approaches to describe the position feature relations, namely, encoding the information in a multi-channel image within the observation range [24], and concatenating the received state information directly [30]. However, both methods have drawbacks. The image information can effectively characterize the relative position relationship, but after pixel discretization, the accuracy of spatial position features is limited, resulting in quantization errors. Although setting a higher image resolution for encoding can reduce discretization errors, it will lead to a nonlinear increase in the computational complexity. In addition, concatenating state information can reduce the computational complexity, but under partially observable conditions, the input dimensionality of neural networks changes dynamically, and the population invariance method must be used to address the problem of observation dimensionality change [21,31]. Based on the second feature description approach, an observational feature network for agents which can be used to deal with dynamic input dimensionality is designed to overcome the drawbacks of this method. The network structure is in subsection 4.4.

The designed observational feature o_i is composed of friendly observational feature o_i^p, evader feature o_i^e, encirclement feature o_i^e and USV state feature o_i^s.

4.1.1. Friendly observational feature

Friendly observational features are employed to describe the state of the friendly USVs within the communication range d_p of the agent. The relationship between agents can be modeled as $\mathcal{G}_\mathcal{P} = (V_p, E_p)$, where $V_p = \{U_1, U_2, \ldots, U_{N_p}\}$, $E_p = \{U_i, U_j | d_{ij} < r_p, i \ne j\}$. The observability between agents can be represented by the connectivity of the graph $\mathcal{G}_p$. For agent i, the information of the USV numbered $\mathcal{N}_{\mathcal{G}}^p(i) = \{m | \{U_i, T_m\} \in E\}$ can be observed, and the friendly observational feature is defined as $o_i^p = \{o_{i,j} | j \in \mathcal{N}_{\mathcal{G}}^p(i)\}$

For agent i, the observational feature of the neighboring agent j is set to $o_{i,j} = \{d_{i,j}, x_{i,j}, y_{i,j}, \Delta v_{i,j}, \Delta v_x, \Delta v_y, Q_{i,j}, Q_{j,i}\}$, where $d_{i,j}$ represents the Euclidean distance between the two agents, $x_{i,j}$ and $y_{i,j}$ are the components of $d_{i,j}$ in the coordinate system, $\Delta v_{i,j} = v_i - v_j$ is the relative velocity difference, Δv_x and Δv_y are the components of $\Delta v_{i,j}$ in the coordinate system, $Q_{i,j}$ and $Q_{j,i}$ are the relative bearing angle between them. Their relationship is shown in Fig. 3.

4.1.2. Observational feature of the evader

Observational features of the evader are to describe the state of the evaders within the observation radius d_e of the USV agent. Similarly, The observation of the target by the agent is modeled as $\mathcal{G}_e = (V, E_e)$, where $V = \{U_1, U_2, \ldots, U_{N_p}, T_1, T_2, \ldots, T_{N_e}\}$, $E_e = \{U_i, T_m | d_{i,m} < d_e, i = 1, 2, \ldots, N_p, m = 1, 2, \ldots, N_e\}$. For the information of the target numbered $\mathcal{N}_{\mathcal{G}}^e(i) = \{m | \{U_i, T_m\} \in E_e\}$ that can be observed by agent i, the observational feature of the evader is

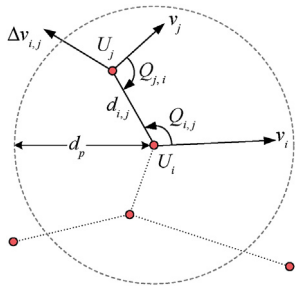

Fig. 3. Schematic of friendly USV observational feature.

defined as $o_i^e = \{o_{i,m} | m \in \mathcal{N}_{\mathcal{G}}^e(i)\}$.

Similarly, for agent i, the observational feature of the evader T_m within the observation radius r_e is set to $o_{i,m} = \{d_{i,m}, x_{i,m}, y_{i,m}, \Delta v_{i,m}, \Delta v_x, \Delta v_y, Q_{i,m}, Q_{j,m}, \kappa(m)\}$, and the definition of the feature element is basically the same as that of the friendly observational feature. The last item $\kappa(m)$ represents the demand function, which refers to the current demand degree of the evader T_m for USVs in the hunting task, $\kappa(m) \in (-1, 1)$. The expression is

$$\kappa(m) = \tanh\left(N_{\text{exp}} - \left|\mathcal{N}_{\mathcal{G}}^m(m)\right|\right) \tag{7}$$

where $\mathcal{N}_{\mathcal{G}}^m(m) = \{i | \{U_i, T_m\} \in E_m\}$ represents the number of USVs involved in hunting the evader T_m, and the dynamic graph $\mathcal{G}_m = (V, E_m)$ is defined, $E_m = \{U_i, T_m | d_{i,m} < d_{cap}, i = 1, 2, \ldots, N_p, m = 1, 2, \ldots, N_e\}$. N_{exp} represents the expected number of USVs involved in the task. When the effective encirclement angle α_{cap} is determined, $N_{\text{exp}} = \lceil 2\pi / \alpha_{\text{cap}} \rceil$.

4.1.3. Encirclement feature

Encirclement features are to describe the optimal position maneuvering trend of the agents in the cooperative state. In the Cartesian coordinate system with the agent as the origin, the orthogonal component can be used to represent the position relationship of the observed targets relative to the agent. This form of position feature can be directly applied to deal with the tracking and formation problems. However, in the capture problem, the agent needs to indirectly understand the cooperative encirclement situation through the relative position information, and then make decisions that facilitate the USVs to form a circular formation around the evader. Obviously, this entails requirements for the strong feature extraction ability of deep networks. In addition, with the same encirclement situation relationship, the position feature is not rotation invariant, which increases the difficulty of network generalization.

To reduce the cost of network learning, this study proceeds from the perspective of feature engineering, extracts the feature of encirclement angle from the encirclement situation, and then decouples it into the Cartesian coordinate components as the encirclement feature, as shown in Fig. 4.

It is assumed that agent i is within the encirclement radius of the evader T_m. If $|\mathcal{N}_{\mathcal{G}}^m(m)| \geq 3$, the encirclement angles of agent i and neighboring USVs are α_{left} and α_{right}, and the average encirclement angle $\alpha_{\text{avg}} = (\alpha_{\text{left}} + \alpha_{\text{right}})/2$ is defined. Let α_{err} be the error angle from agent i to the angular bisector, and let $\boldsymbol{\lambda}_{\text{err}} \propto \alpha_{\text{err}}$ represent the tangential error of U_i relative to T_m, which is taken as the observational error of agent i encirclement, $\boldsymbol{\lambda}_{\text{err}} = \{\Delta x_i, \Delta y_i\}$. Δx_i and Δy_i are the Cartesian coordinate components of $\boldsymbol{\lambda}_{\text{err}}$, which reflects the reference error of the tangential component of the target encirclement situation formed by agent i, and the encirclement feature $o_i^c = \boldsymbol{\lambda}_{\text{err}} = \{\Delta x_i, \Delta y_i\}$ is set.

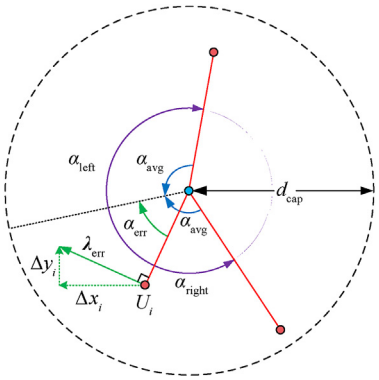

Fig. 4. Schematic of encirclement feature.

4.1.4. USV state feature

USV state features describes the state attributes of the agent itself. $o_i^s = \{v_x, v_y, a_x, a_y\}$, including the current speed v_x and v_y of the USV and the action output a_x and a_y at the previous time. The action design is detailed in Section 4.3.

4.1.5. Observational features

To sum up, the observational features o_i of agent i are expressed as

$$o_i = \left\{o_i^p, o_i^e, o_i^c, o_i^s\right\}$$

$$\begin{aligned} o_i^p &= \{o_{i,j} | j \in \mathcal{N}_{\mathcal{G}}^p(i)\} \\ o_i^e &= \{o_{i,m} | m \in \mathcal{N}_{\mathcal{G}}^e(i)\} \\ o_i^c &= \{\Delta x_i, \Delta y_i\} \\ o_i^s &= \{v_x, v_y, a_x, a_y\} \end{aligned} \tag{8}$$

In the formulas, the number of elements of the friendly observational feature o_i^p and of the target observational feature o_i^e changes dynamically, which is related to the spatial location and observation range of the agent. The encirclement feature o_i^c and the USV state feature o_i^s are one-dimensional vectors with fixed length.

4.2. Reward function design

Based on target capture in Section 3.1, the current state of the agent can be effectively evaluated by designing an appropriate reward function, which combines guiding rewards and sparse rewards. Whenever an evader is captured, the agent participating in the capture will get a one-time reward. During task execution, each agent will receive a guiding reward at each time step, which aims to guide the agents to chase the evader and cooperate to form an encirclement situation. Finally, the average reward of all agents is calculated as the shared reward of the USV fleet.

The reward function R consists of capture reward, guiding reward, collision penalty and time consumption penalty by weight, which is defined as

$$R = \frac{1}{N_p} \sum_i \sum_k r_k(i) \tag{9}$$

where $r_k(i)$ is the k-th reward of agent i. Since agents are homogeneous, r_k is used to uniformly refer to $r_k(i)$ in the rest of the paper.

4.2.1. Capture reward r_1

$$r_1 = \frac{\alpha_{\text{left}} + \alpha_{\text{right}}}{4\pi} r_{\text{cap}} e^{-k_1 \sigma(\alpha)} \tag{10}$$

When an evader is successfully captured, the agents participating in the capture will get a reward according to their contribution. The criteria for contribution evaluation is the proportion of the encirclement angles between the agent and the target (i.e. half of the encirclement angle adjacent to the left and right of the agent), expressed by $\left(\alpha_{\text{left}} + \alpha_{\text{right}}\right)\big/4\pi$. r_{cap} is the coefficient of total capture rewards. To evaluate the circular formation around the target, it is considered that the more consistent the encirclement angles α formed, the more complete the circular formation of the agents around the target. Thus, a reward function coefficient $e^{-k_1\sigma(\alpha)}$ in the form of a negative exponent is established, where $\sigma(\alpha)$ is the standard deviation of the encirclement angle and k_1 is the accommodation coefficient.

4.2.2. Guiding reward r_2

$$r_2 = \begin{cases} r_{\text{guide}} & , d_{\text{tar}} \le d_{\text{cap}} \\ r_{\text{guide}} e^{-k_2 d_{\text{tar}}} & , d_{\text{tar}} > d_{\text{cap}} \end{cases} \tag{11}$$

To guide the agents to approach the evaders in the early stage of training, the agents will receive positive rewards related to the distance to the targets in each time step. In the formula, $d_{\text{tar}} = \min_m d_{i,m}$ represents the distance between an a gent and its nearest target. When $d_{\text{tar}} > d_{\text{cap}}$, the agent enters the enclosed area and obtains a guiding reward $r_2 = r_{\text{guide}}$. When $d_{\text{tar}} > d_{\text{cap}}$, the guiding reward coefficient r_{guide} decreases exponentially with the increasing distance, where $r_2 = r_{\text{guide}} e^{-k_2 d_{\text{tar}}}$ and k_2 is the accommodation coefficient.

4.2.3. Collision penalty r_3

$$r_3 = -r_{\text{crash}} e^{-k_3 d_{\text{agent}}} \tag{12}$$

A negative exponential reward function r_3 is established to describe the collision risk between agents, where r_{crash} is the collision penalty coefficient, $d_{\text{agent}} = \min_j d_{i,j}$ represents the shortest distance between agent i and other agents, and k_3 is the accommodation coefficient.

4.2.4. Time consumption penalty r_4

$$r_4 = -r_{\text{time}} \tag{13}$$

To ensure the timeliness of tasks, the agent will receive a negative return r_{time} in each time step.

4.3. Action space design

For kinematic problems, the action space is usually designed as the control input of kinematic systems. The kinematic model of holonomic motion constraints outputs the acceleration vector directly from the policy network [17,30]. The kinematic model of non-holonomic motion constraints of UAVs, UGVs and other unmanned systems can be simplified into a first-order system so as to determine the controlled quantities of the policy network's output speed and angular velocity [18,32]. Compared with UAVs and UGVs, the control response of USVs is slow, and thus usually described by a second-order system. The controlled quantities are acceleration a_v and angular acceleration a_ω. However, a_v and a_ω have a complex coupling relationship with the reward function R. When a_v and a_ω are directly used as network output, it is difficult to guarantee the learning efficiency and convergence of the network.

Considering that the designed observational features are mainly orthogonal components of the coordinate, the action space $\boldsymbol{a}$ designed in this paper is set as the two orthogonal independent controlled quantities in the x and y directions, namely, $\boldsymbol{a} = \{a_x, a_y\}$ and $a_x, a_y \in [-1, 1]$, and then the mapping between action space $\boldsymbol{a}$ and the expected speed v_{exp}/expected direction ψ_{exp} of the USV is established:

$$\begin{cases} v_{\text{exp}} = v_{\max} \max\left(|a_x|, |a_y|\right)^2 \\ \varphi_{\text{exp}} = \arctan\left(a_x / a_y\right) \end{cases} \tag{14}$$

where $v_{\max}$ is the maximum USV speed. Finally, the USV tracking control algorithm [33] is introduced to indirectly control a_v and a_ω so that the USV navigates according to the expected navigation parameters.

By avoiding the involvement of the policy network in the complex coupling relationship between the kinematic control and reward function of the USV, the proposed method reduces the decision-making difficulty of the system.

4.4. Network structure design

Due to the partial observability of agents, the number of friendly agents and evaders in the observation range changes dynamically, leading to the uncertainty of the observed dimension features. Thus, it is necessary to design a network structure applicable to the uncertain dimension feature input. In this paper, a feature embedding block based on learning is designed first, and then the structures of the policy network and value network are designed respectively.

4.4.1. Design of feature embedding block

The idea of feature embedding is to encode a variable number of homogeneous state features into features with fixed dimensions. Ref. [21] proposed the method of using neural networks to define the feature space of a mean embedding, which has better information coding performance than the histogram-based method [34] and radial basis functions (RBF) method [35]. Based on Ref. [21], we construct a feature embedding block (FEB) applicable to observational features of agents. FEB is designed to apply a network layer with parameter learning capability to each item in the two-dimensional observational features of agents, and then further compress the output two-dimensional features into one-dimensional features with fixed dimensions. Inspired by the pooling layer in convolutional neural networks, we propose two feature compression methods, maximum pooling and average pooling, and realize feature dimension reduction for the first time. The structure of FEB is presented in Fig. 5.

FEB inputs the observational features. Firstly, the same type of attributes of different items are standardized, so that the observations such as angle and distance are on the same quantitative scale. Then, each item passes through the fully connected layer with the same weight to obtain the features of the same dimension. To improve network robustness, the dropout operation can be added in the last step to randomly mask some observational features. Two feature compression methods, column-wise max-pooling (CMP) and column-wise average-pooling (CAP), are designed. Finally, FEB passes through the CMP or CAP layer to obtain the embedded features.

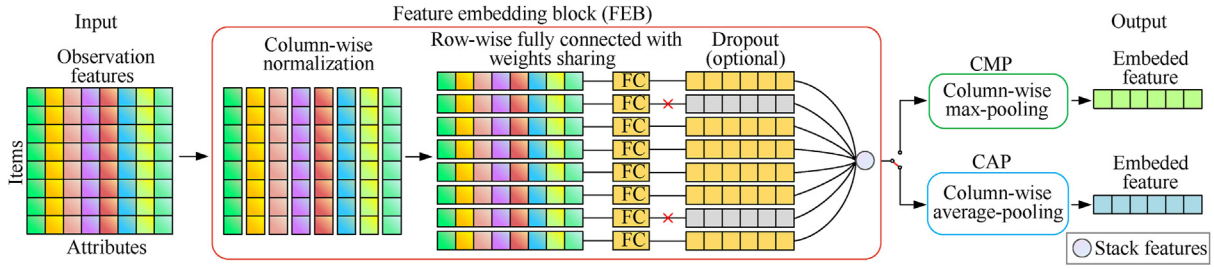

Fig. 5. Structural diagram of feature embedded block (FEB).

4.4.2. Policy network structure design

The policy network π_θ inputs the observational feature o^i of the agent as described in Section 4.1. Firstly, the friendly agent's observational feature o_i^p and the evader's observational feature o_i^e input two independent FEB respectively. The dropout technique is used for the FEB passed by o_i^p, which is then processed by the CMP and CAP layers, and four embedded features with 32 dimensions are obtained. Subsequently, they are connected with the encirclement feature o_i^c and the USV state feature o_i^s to form a feature vector with a length of 134. Finally, after they are calculated by the two-layer Fully Connected (FC) network and Gated Recurrent Unit (GRU) network respectively, the mean of the Gaussian distribution $\boldsymbol{\mu} = [\mu_x, \mu_y]$ and standard deviation $\boldsymbol{\sigma} = [\sigma_x, \sigma_y]$ of the controlled quantity $\boldsymbol{a} = \{a_x, a_y\}$ are output. During training, the controlled quantity can be obtained by sampling the Gaussian distribution. In the evaluation, $a_x = \mu_x$, $a_y = \mu_y$. The policy network structure is shown in Fig. 6.

4.4.3. Value network structure design

The value network V_φ inputs the set of observations of all agents $\boldsymbol{o} = [\boldsymbol{o}^p, \boldsymbol{o}^e, \boldsymbol{o}^c, \boldsymbol{o}^s]$. The elements in $\boldsymbol{o}$ are the vertical stacking of each agent's corresponding elements, $\boldsymbol{o}^c$ and $\boldsymbol{o}^s$ represent the two-dimensional features of the N_p row. Different from the policy network, the designed value network concatenates $\boldsymbol{o}^c$ and $\boldsymbol{o}^s$ horizontally, and after the processing of the feature embedding block, the embedded features with a length of 64 are output. The middle layer structure of the network is consistent with π_θ, and finally the network outputs the predicted value of the state value Q. The value network structure is shown in Fig. 7.

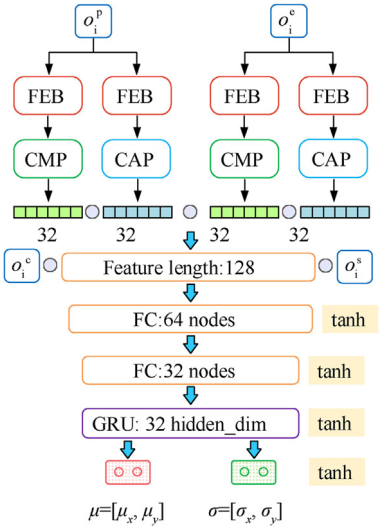

Fig. 6. Schematic of policy network structure.

4.5. Algorithm flow design

As each agent in the multi-agent system is affected by the environment and other agents simultaneously, the single-agent RL algorithm cannot produce ideal effects, and thus it is necessary to use joint state-action space to train the agents. To this end, some centralized training and decentralized execution methods such as MADDPG [30], COMA [36], QMIX [37], and MAPPO [38] have been proposed. Among them, MAPPO (Multi-agent Proximal Policy Optimization) is a variant of Proximal Policy Optimization (PPO) [26] applied to multi-agent tasks. It is an algorithm with the actor-critic architecture and has high learning efficiency under the condition of limited computing resources. It can deliver the performance of the mainstream MARL algorithm by performing only a small amount of hyperparameter search [38], and has demonstrated high effectiveness in a variety of cooperative tasks. Therefore, we use MAPPO as the learning method for the multi-USV cooperative hunting problem.

Based on the PPO algorithm, MAPPO is extended to the scenario of multi-agents by training two independent networks, namely, the policy network π_θ with parameter θ and the value network V_φ with parameter φ. They share network weight for all homogeneous agents. The policy network π_θ maps the local observation o^i of agent i to the mean and standard deviation of the continuous action space. The value network $V_\varphi : \mathscr{S} \rightarrow \mathbf{R}$ maps the global state and the state action of agent i to the estimate of the current value Q. In the case of multiple agents, the loss function $L^{CLIP}(\theta)$ is rewritten as

$$L^{CLIP}(\theta) = \frac{1}{BN}\sum_{i=1}^{B}\sum_{k=1}^{N}\min\left[r_{\theta,i}^{(k)}A_i^{(k)}, \mathrm{clip}\left(r_{\theta,i}^{(k)}, 1-\varepsilon, 1+\varepsilon\right)A_i^{(k)}\right] \quad (15)$$

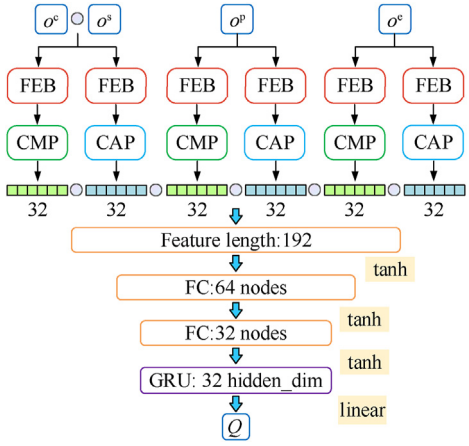

Fig. 7. Schematic of value network structure.

where B is the number of batch processing, N is the number of USV in the fleets, $r_{\theta,i}^{(k)} = \frac{\pi_\theta(a_i^{(k)}|s_i^{(k)})}{\pi_{\theta_{old}}(a_i^{(k)}|s_i^{(k)})}$, which is the importance sampling weight in the form of multiple agents, $A_i^{(k)}$ is the corresponding advantage function.

Accordingly, the loss function $L^{CLIP}(\varphi)$ of the value network is

$$L^{CLIP}(\varphi) = \frac{1}{BN}\sum_{i=1}^{B}\sum_{k=1}^{N}\max\left[\left(V_\varphi\left(s_i^{(k)}\right) - R_i\right)^2, \left(\text{clip}\left(V_\varphi\left(s_i^{(k)}\right), V_{\varphi_{old}}\left(s_i^{(k)}\right) - \varepsilon, V_{\varphi_{old}}\left(s_i^{(k)}\right) + \varepsilon\right) - R_i\right)^2\right] \tag{16}$$

In the present paper, the centralized learning and decentralized execution (CLDE) framework is used to implement the multi-target hunting by USVs. In the training process, a centralized control center is constructed for obtaining the global state information, which is used to train the agents. In the evaluation process, each agent only needs to make action execution decisions based on its observations, without considering the states and actions of other agents. The framework flowchart is in Fig. 8.

During training, the parallel training environment outputs the global state $\boldsymbol{s}_t$, and then the reward function R and the observation model O output the reward r and observation o obtained by each USV in this step of training according to $\boldsymbol{s}_t$. The USV policy network outputs action a according to the observation o. Finally, the parallel training environment inputs the joint action $\boldsymbol{a}_t$ of each USV and gives the updated global state $\boldsymbol{s}_{t+1}$, which means completing an interactive cycle. During each cycle, the intermediate state data is stored in the experience replay buffer in the form of $[\boldsymbol{s}_t, \boldsymbol{o}_t, \boldsymbol{h}_{t,\pi}, \boldsymbol{h}_{t,V}, \boldsymbol{a}_t, \boldsymbol{r}_t, \boldsymbol{s}_{t+1}, \boldsymbol{o}_{t+1}]$, where $\boldsymbol{h}_{t,\pi}$ and $\boldsymbol{h}_{t,V}$ are the hidden state of Recurrent Neural Network (RNN) in the policy network and value network respectively. When the collected experience reaches the specified capacity, the updated gradient of the policy network and value network is calculated, and the network weights are updated with the Adam optimizer.

Therefore, the testing process is relatively simple. The observation model O outputs the observation o of each USV according to the global state $\boldsymbol{s}_t$, then the USV outputs action a based on the observation o, and finally the test environment updates the global state according to the joint action $\boldsymbol{a}_t$, which is a complete cycle.

The pseudocode of the algorithm is as follows.

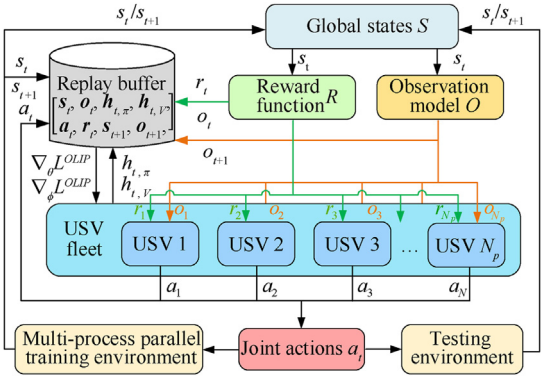

Fig. 8. Centralized training and decentralized execution framework of USV fleet.

Algorithm 1. DPOMH-PPO algorithm

Algorithm 1 DPOMH-PPO algorithm

Input:

Rollout steps E, batch processes number B, the maximum steps in an episode T, the chunks of length for RNN L, the learning rate l_r, and the times of gradient updates K.

Output:

Policy network parameter θ, value network parameter ϕ.

Parameters of policy network π and value network V are initialized to θ and ϕ, respectively.
while *step* $\le E$ **do**
  Set an experience replay buffer $D=\{\}$
  for i=1 **to** B **do**
    $\tau=[]$ empty list
    Initialize USV capture environment
    Initialize RNN network status $\boldsymbol{h}_{0,\pi}$ for actor
    Initialize RNN network status $\boldsymbol{h}_{0,V}$ for critic
    for t=1 **to** T **do**
      for all agent j **do**
        $p_t^{(j)}, h_{t,\pi}^{(j)} = \pi(o_t^{(j)}, h_{t-1,\pi}^{(j)}; \theta)$
        $a_t^{(j)} \sim p_t^{(j)}$
        $v_t^{(j)}, h_{t,V}^{(j)} = V(s_t^{(a)}, h_{t-1,V}^{(a)}; \phi)$
      end for
      Executes action $\boldsymbol{a}_t$ and obtain $\boldsymbol{r}_t$, $\boldsymbol{s}_{t+1}$, $\boldsymbol{o}_{t+1}$
      τ += $\left[\boldsymbol{s}_t, \boldsymbol{o}_t, \boldsymbol{h}_{t,\pi}, \boldsymbol{h}_{t,V}, \boldsymbol{a}_t, \boldsymbol{r}_t, \boldsymbol{s}_{t+1}, \boldsymbol{o}_{t+1}\right]$
    end for
    Calculate the advantage estimate A and cumulative discounted reward R on τ
    Split trajectory τ chunks of length L
    for l=0,1,…, $T//L$ **do**
      $D = D \cup (\tau[l:l+L], \hat{A}[l:l+L], \hat{R}[l:l+L])$
    end for
  end for
  Calculate the gradient $\nabla_\theta L^{CLIP}$ and use the Adam optimizer to update the policy network π_θ for K times at learning rate l_r
  Calculate the gradient $\nabla_\phi L^{CLIP}$ and use the Adam optimizer to update the value network V_ϕ for K times at learning rate l_r
end while

5. Simulation experiment and analysis

In this section, a multi-target hunting simulation environment is built, the USV policy network is trained with the proposed DPOMH-PPO algorithm, and its effectiveness is discussed. Finally, the performance, generalization and robustness of the algorithm are compared and analyzed.

5.1. Experimental environment and parameter setting

The experiment is performed on a workstation with i9-10980XE CPU, RTX2080Ti × 4 GPU(11 GB × 4 VRAM), and 128 GB RAM.

Pytorch is used as the deep learning training framework and OpenAI Gym as the reinforcement learning environment framework. A multi-agent runtime environment is built for the cooperative multi-target hunting problem, and the Gym API is redefined.

In particular, the experimental conditions are set as follows:

1) The motion parameters and observation performance of all USVs are consistent, and the escape strategy and motion parameters of all evaders are the same.

Since there is no geographical boundary constraint in the marine environment, a boundary-free experimental environment is established. To ensure the existence of feasible solutions, only the pursuit-evasion scenario of high-speed hunting USVs and low-speed evaders is considered.

2) At the beginning of each experiment, the USVs are set into an equally spaced formation. In order not to lose generality, the starting position of the target and the orientation of the USVs relative to the target are randomly generated. The schematic of the initial state of the environment is shown in Fig. 9.
3) Simulation environment parameters are set as shown in Table 1.
4) After multiple experiments and parameter tuning, the parameters of the reward function in DPOMH-PPO are determined as shown in Table 2
5) During network training, curriculum learning is employed to alleviate the sparse reward problem. First, we start the training with a simple hunting task, and then gradually increase the task difficulty to the level of actual difficulty. The factors determining the difficulty of the hunting problem include the speed of the evader and the encirclement radius d_{cap}. In the early stage of training, the target speed is reduced and the encirclement radius is increased so that the USVs can easily obtain rewards and have sufficient exploration opportunities. Such a method facilitates USVs to perform more complex cooperative behaviors. After multiple experiments and hyperparameter tuning, the hyperparameter setting for the training of DPOMH-PPO is determined as shown in Table 3

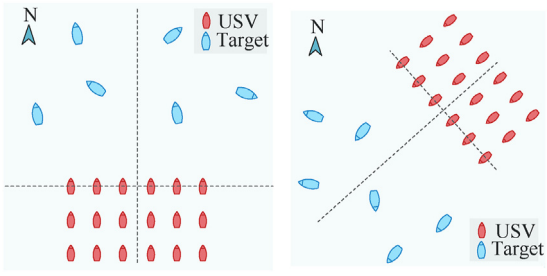

Fig. 9. Schematic of starting position in the simulation environment.

Table 2
Setting of reward function parameters.

Parameter	Value
Coefficient of total capture rewards r_{cap}	100
Guiding reward coefficient r_{guide}	0.5
Collision penalty coefficient r_{crash}	0.5
Time consumption penalty coefficient r_{time}	0.2
Accommodation coefficient k_1, k_2, k_3	0.03, 0.005, 0.08

Table 3
DPOMH-PPO hyperparameter setting.

Parameter	Value
Rollout steps E	4×10^6
batch processes number B	8
Episode length T	400
RNN sequence length L	10
Experience relay buffer capacity R	3200
Gradient update times K	10
Discount factor γ	0.99
Clipping coefficient ε	0.2
Learning rate l_r	0.00005
Optimizer	Adam

5.2. Effectiveness analysis of DPOMH-PPO

5.2.1. Effectiveness analysis for different numbers of USVs

This section evaluates the effectiveness of DPOMH-PPO under different initial USVs numbers. Since the state feature design is scalable, theoretically, only one USV number is needed for training, and then the weight of its policy network can be compatible with any number of USVs in the cooperative hunting tasks. However, to make comparisons of algorithm performance, the number of USVs is set to 6, 8, 10, …, 24 (the number of targets is set to be half of the number of USVs), and the training is completed independently for each case, which corresponds to the optimal strategies in 9 cases respectively. Theoretically, the number of USVs can be larger, but considering the VRAM limit, the maximum number of USVs for training is 24.

To ensure the reliability of the results, we use different random seeds to train each case for 5 times, and the number of training steps in each experiment is 6 million. Among them, the reward curves for three hunting scenarios, namely, 6 USVs encircling 3 targets, 12 USVs encircling 6 targets, and 24 USVs encircling 12 targets (hereinafter abbreviated as N_p vs N_e, representing the number of USVs and targets) are shown in Fig. 10.

As can be seen from Fig. 10, the average reward per step increases rapidly in the first 1 million steps, then gradually stabilizes and converges after 2 million steps. When the training is finished, it can be observed that the average reward per step decreases as the problem scale increases. This is due to the increased number of USVs leading to a larger average distance between the USV fleet and the evader at the initial time, thus increasing time consumption. In

Table 1
Parameters setting for simulation environment.

Environment parameter	Value	Environment parameter	Value
Number of USVs involved in hunting task	6 to 24	Time step/s	0.5
Number of evaders	3 to 12	Episode time limit/s	200
Encirclement radius/m	30	Effective encirclement angle	π
Agent communication range/m	400	Agent observation radius/m	600
USV spacing at the initial position/m	50	Target initial position range/m	200×400
USV initial speed/(m·s^{-1})	5	Target initial speed/(m·s^{-1})	3
USV maximum speed/(m·s^{-1})	10	Target maximum speed/(m·s^{-1})	5
USV maximum angular velocity/(rad·s^{-1})	0.5	Target maximum angular velocity/(rad·s^{-1})	0.2

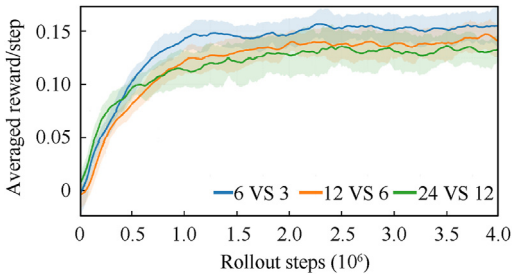

Fig. 10. Reward curves of DPOMH-PPO algorithm in different scenarios.

addition, the USVs are constrained by partial observability and can only rely on local information to make decisions. With the increase of the number of pursuers and evaders, the agents are distributed in a wider spatial range, and the information obtained by the USV becomes more incomplete, so the quality of decision-making outcomes is reduced.

Figs. 11–13 shows the sample data records of 6 VS 3, 12 VS 6, and 24 VS 12 scenarios. The red and blue vessels in the figures represent the USVs and targets respectively. The red and blue tapered lines stand for the trajectories of the USVs and targets respectively, which reflect the temporal relationship of the trajectories of the two sides. When any evader meets the hunting conditions (the USVs enter the dotted line area and the effective encirclement angle is less than π), the target is incapacitated (shown in gray in the figure). When all the targets are successfully encircled, the task is finished immediately.

From the trajectories in Fig. 11(a), it can be found that the USVs can preferentially select their nearest targets ($T = 45$ s) and cooperate to encircle the target. When it is successfully captured, the USVs will immediately turn to pursing the next target ($T = 60$ s). From the trajectories of the USVs in Fig. 11(b), it is found that the USVs have learned to surpass the target from the rear side to intercept the target and force it to turn ($T = 85$ s), thus creating a good opportunity for the subsequent encircling by friendly USVs.

Observing Figs. 12 and 13, we see that facing more complex

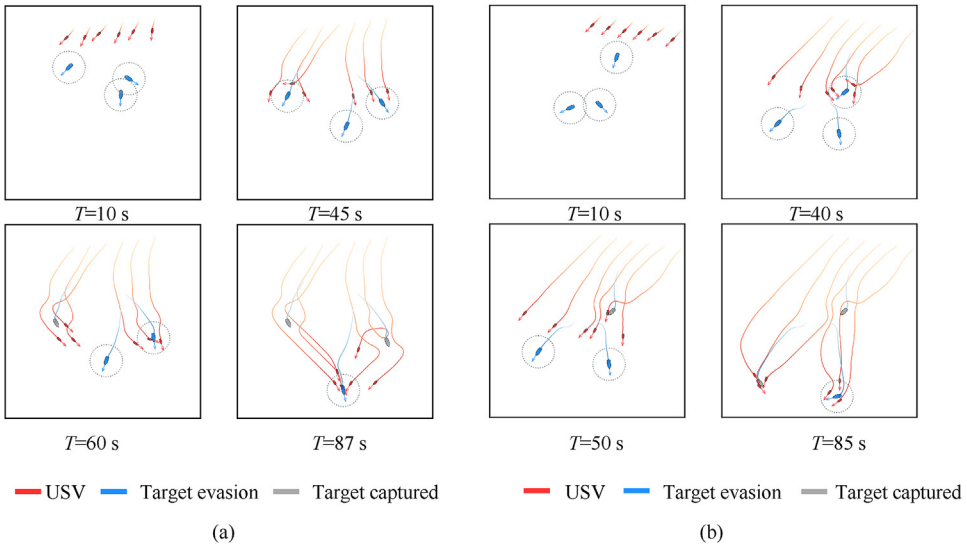

Fig. 11. 6 USVs vs 3 Targets hunting process: (a) Test scenario 1; (b) Test scenario 2.

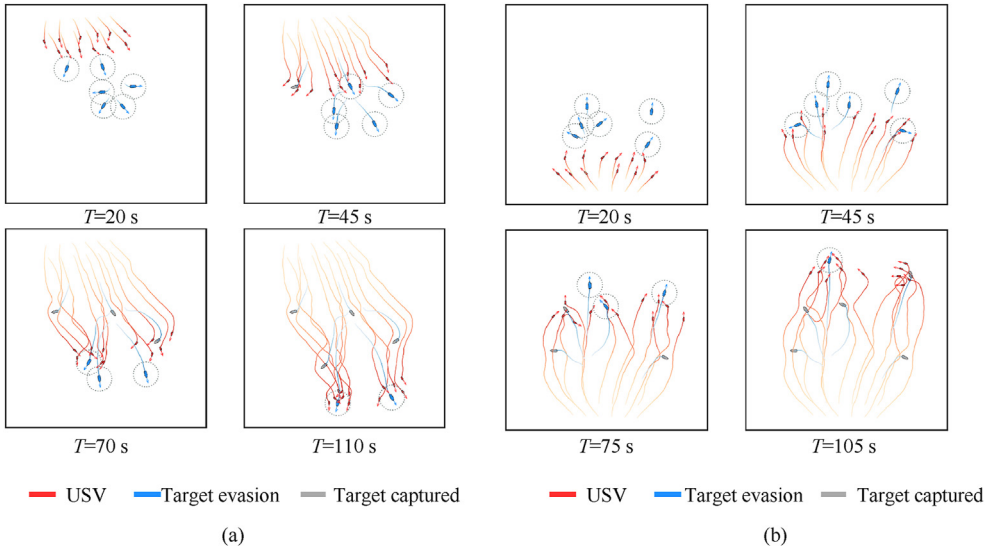

Fig. 12. 12 USVs vs 6 Targets hunting process: (a) Test scenario 1; (b) Test scenario 2.

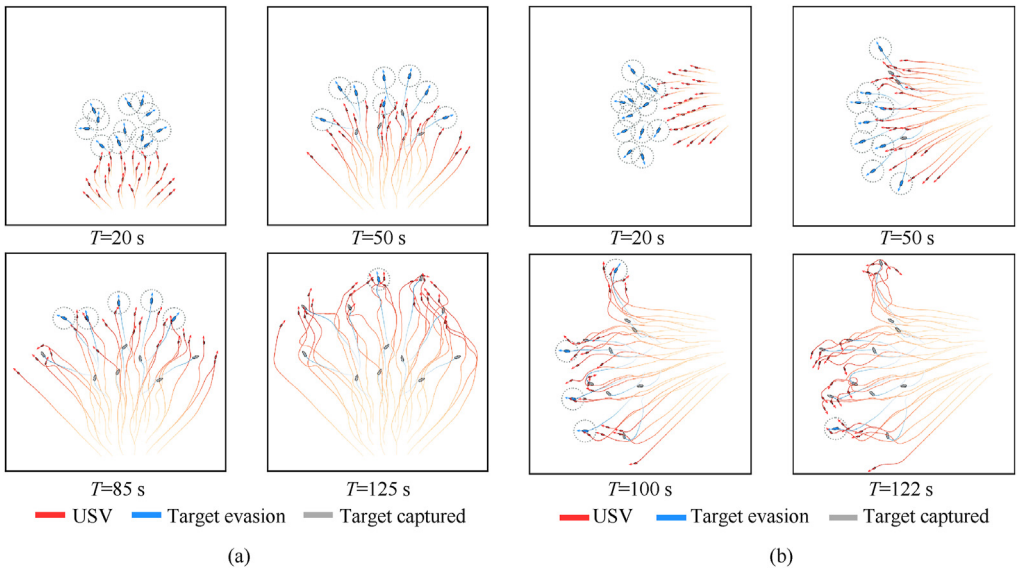

Fig. 13. 24 USVs vs 12 Targets hunting process: (a) Test scenario 1; (b) Test scenario 2.

situations, the USV fleet can still successfully capture all the targets through cooperation. With the increase of the problem scale, the time required for capture increases as well. DPOMH-PPO can achieve the cooperative hunting of multiple targets by USV fleets under the scenarios of different numbers of USVs and targets.

5.2.2. Ablation experiment

To investigate the effects of the feature embedding block (FEB) and network structure parameters on the performance of DPOMH-PPO, an ablation experiment is designed. Based on the network structure designed originally, the contribution of each function to the algorithm's performance is compared by masking certain structures or adjusting the parameters of the networks. The experimental scenario is set to 12 USVs vs 6 Targets, and the experimental results are listed in Table 4. It can be concluded that the dropout method, CMP and CAP layers involved in the FEB can improve the algorithm's performance, of which the CAP layer has the most notable contribution, with an ablation loss of 11.64%; the contribution of curriculum learning accounts for 8.22%; halving the feature size will reduce the average reward per step by 9.59%, and conversely, it will be increased by 7.53%; although increased feature size leads to improved rewards, the network training time and computational complexity will be doubled as well. Ablation experiments have further verified the effectiveness of the proposed DPOMH-PPO algorithm.

Table 4
Results of ablation experiments.

No.	Ablation item	Average reward/step	Loss contribution/%
1	Baseline	0.146	0.00
2	No dropout	0.141	−3.42
3	No CMP	0.136	−6.85
4	No CAP	0.129	−11.64
5	No curriculum learning	0.134	−8.22
6	Half feature size	0.132	−9.59
7	Double feature size	0.157	7.53

5.3. Comparative analysis of methods

5.3.1. Performance comparison of algorithms

In this study, three other algorithms are also selected to compare with DPOMH-PPO, including the IL (independent learning) version of DPOMH-PPO, histogram method [34] and Minimap method [24]. The IL method uses the traditional single-agent PPO algorithm, where the input of the value network and the policy network is the same, and other parameters are the same. The histogram method maps the observed target position to the range-azimuth 2D mesh, where the mesh size and distance resolution are set to 20 × 24 and 20 m/unit mesh. By referring to the spatial position representation in the MAgent environment, the Minimap method maps target observations to Minimap images with USV as the origin, where the image size and distance resolution are set to 21 × 21 pixels and 40 m/pixel. These methods have completed training in 9 test environments with the number of USVs ranging from 6 to 24. Then the algorithms' performance is evaluated from two aspects, average reward per step and average task completion time. Using the average reward per step, the algorithm's performance can be directly compared, and the larger the average reward per step, the better the performance. The average task completion time reflects the algorithm's efficiency in performing the cooperative hunting task, and the less time the algorithm takes, the better the performance.

Fig. 14 shows the reward curves of 6 VS 3, 12 VS 6, and 24 VS 12 respectively. As illustrated, all methods can converge within the training steps, and the average reward per step of DPOMH is the highest in all scenarios; the performance of the IL method is close to that of DPOMH but the gap between them obviously widens as the problem scale increases, which can be explained by the fact that independent learning cannot obtain joint state information, so the variance of the gradient in value network training becomes larger; in term of the average reward per step, Minimap and Histogram is inferior to DPOMH, and in terms of convergence speed, Minimap is faster than Histogram, but at the end of training, Histogram's performance slightly oversteps that of Minimap.

Fig. 15 further shows the variation of average reward per step and average task completion time of the four algorithms under different initial numbers of USVs. As can be seen from Fig. 15(a), with the increasing number of USVs, the average reward per step of the DPOMH and IL methods decreases slowly, while that of the other two methods increases gradually; the average reward per step of DPOMH is always better than that of the other methods,

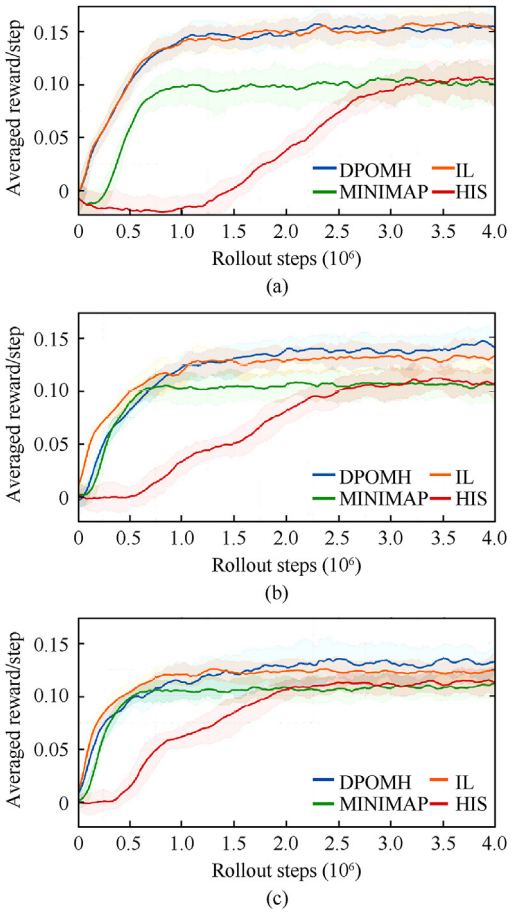

Fig. 14. Comparison of reward curves of training of different multi-target hunting algorithms: (a) 6 VS 3; (b) 12 VS 6; (c) 24 VS 12.

Table 5
Task completion time of different algorithms with USVs failed (Unit: s).

Number of failed USVs	DPOMH-PPO	IL-PPO	HIS-PPO	Minimap-PPO
0 (baseline)	103.77	114.88	124.92	121.93
1	127.64	147.38	171.66	149.28
2	156.11	174.66	199.98	178.07
3	189.05	204.19	216.48	197.38
4	218.29	220.98	237.38	221.65
5	228.80	234.87	274.24	241.08
6	235.20	246.12	271.07	264.38

which is consistent with the results shown in Fig. 14. Fig. 15(b) presents the change in the task completion time with the number of USVs. As the number of USVs increases, the time taken by the four methods all increases monotonously, and DPOMH completes the task in the shortest time. It can be found that the task completion time is negatively correlated with the average reward per step, which can be explained by the fact that long task time means more time penalties received by agents.

5.3.2. Generalization performance analysis

To further investigate the multi-scenario migration capability of DPOMH-PPO, the method adopted in this study is to migrate the learned hunting strategy to a larger test environment than during training, and then the changes in performance indexes such as average reward per step and average task completion time are analyzed and compared. In the experiments, the optimal strategies of the four methods for the scenario of 12 VS 6 are selected and then replicated for deployment in the test scenarios of 24 VS 12, 36 VS 18, and 48 VS 24 respectively. The comparison of performance indexes is presented in Fig. 16. As shown in Fig. 16(a), as the problem scale doubles, the average reward per step gradually decreases, but DPOMH-PPO performs better than the other methods in each scenario, and the performance of HIS decreases most

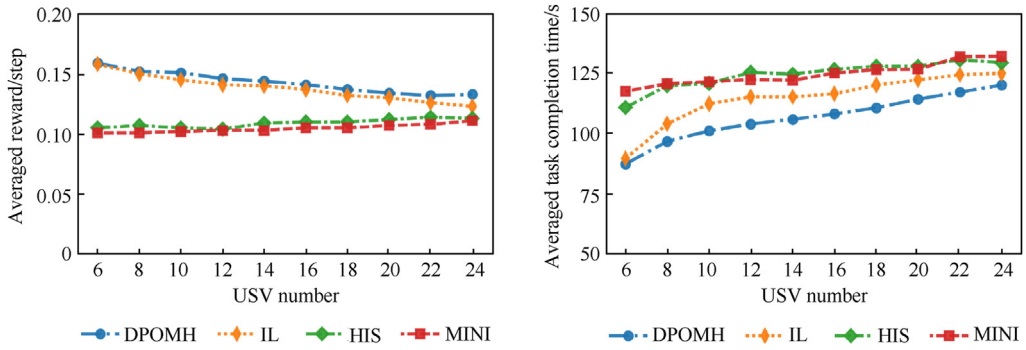

Fig. 15. Comparison of performance indexes of different algorithms: (a) Average reward per step; (b) Average task completion time.

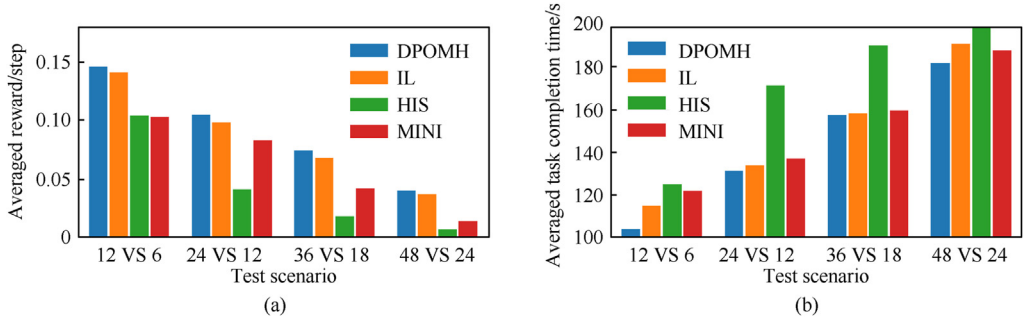

Fig. 16. Comparison of generalization performance of different algorithms: (a) Average reward per step; (b) Average task completion time.

significantly. The shortest task completion time is seen in the DPOMH-PPO as shown in Fig. 16(b), indicating that DPOMH-PPO possesses better generalization performance.

5.3.3. Robustness analysis

To analyze the robustness of the algorithms, the damage of USVs is simulated by means of node failure. Specifically, USVs are randomly selected every other period in the hunting process; the throttle and rudder angle control are set to 0, and the observability of the damaged USV is blocked. In this way, the self-organization and flexible coordination capabilities of the USV fleet in the case of suffering damage can be verified. The experiment takes the 12 VS 6 test scenario as the baseline. From the beginning of the task, one USV in the fleet is randomly selected to fail every 15 s until the total number of failed USVs reaches the preset loss limit.

The averaged time consumption in various algorithms for USVs failed in different cases is shown in Table 5. It can be observed that with the increasing number of failed USVs, the task completion time of the four methods increases significantly, and that the time taken by DPOMH-PPO is the shortest under the same conditions.

The reward and efficiency of different algorithms are listed in Table 6. The step reward refers to the average reward per step obtained by the active USVs. The efficiency is defined as the ratio of the current step reward to the baseline step reward, which represents the performance of the method after certain USVs have failed. Under the same number of removed agents, the higher efficiency of the algorithm means stronger robustness. As can be seen from Table 6, DPOMH-PPO has the highest efficiency and HIS-PPO has the lowest efficiency under different number of failed USVs. Combining Tables 5 and 6, we find that DPOMH-PPO has the strongest robustness.

The record of the sample data of DPOMH-PPO during the hunting process with 6 USVs failed is shown in Fig. 17. In the figure, the gradually changing trajectories of each agent are retained for 100 s, the yellow USV indicates that it is in the inactive state, and the deactivation time of the USV (T = 15 s, 30 s, …, 90 s) is marked

Table 6
Step reward and efficiency of different algorithms under different number of failed USVs.

Number of Removed USVs	DPOMH-PPO		IL-PPO		HIS-PPO		Minimap-PPO	
	Step reward	Efficiency	Step reward	Efficiency	Step reward	Efficiency	Step reward	Efficiency
0 (baseline)	0.146	100.00%	0.141	96.58%	0.104	71.23%	0.100	70.55%
1	0.134	91.78%	0.139	95.21%	0.078	53.42%	0.091	62.33%
2	0.120	82.19%	0.116	79.45%	0.065	44.52%	0.080	54.79%
3	0.113	77.40%	0.107	73.29%	0.063	43.15%	0.079	54.11%
4	0.104	71.23%	0.102	69.86%	0.062	42.47%	0.078	53.42%
5	0.102	69.86%	0.101	69.18%	0.061	41.78%	0.076	52.05%
6	0.100	68.49%	0.097	66.44%	0.058	39.73%	0.070	47.95%

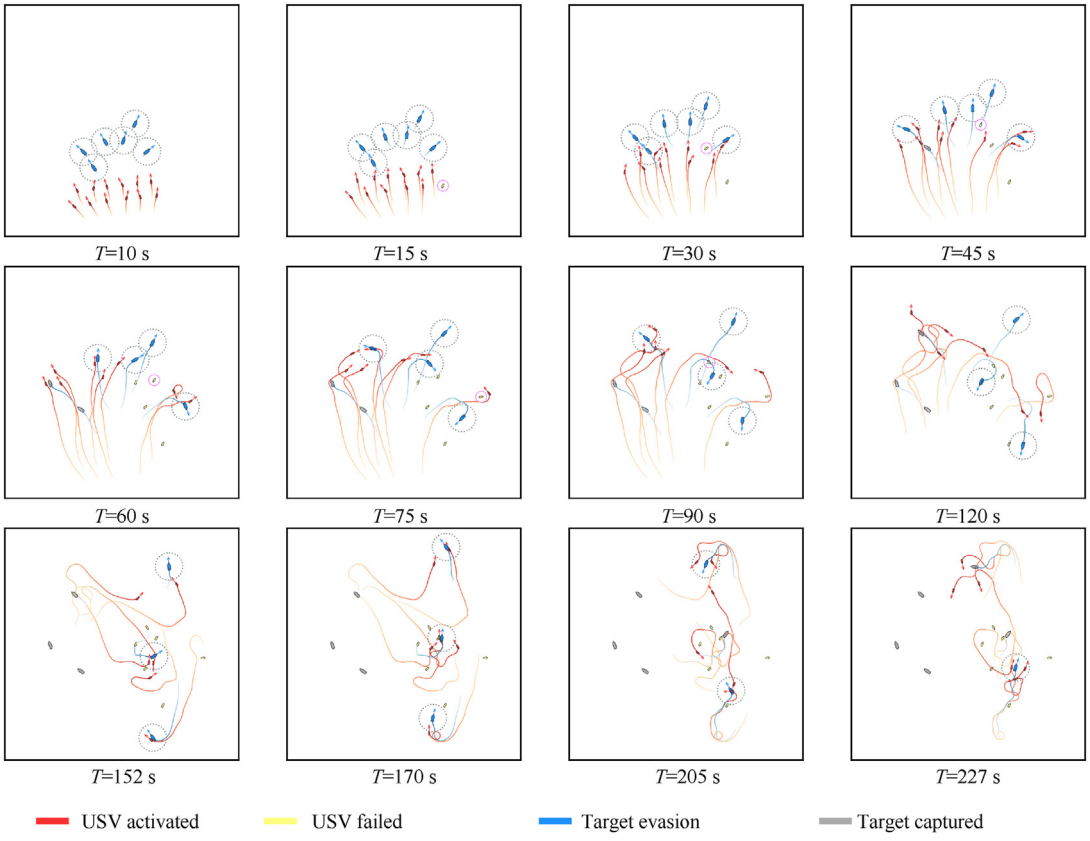

Fig. 17. DPOMH-PPO's multi-target hunting process in the case of 6 USVs failed.

by a purple circle. From Fig. 17, we can see that when a friendly USV is inactivated, the other USVs actively gather ($T = 90$ s), and then reselect targets ($T = 120$ s). Three USVs in the middle encircle the target in the middle nearby ($T = 152$ s, 170 s); two USVs navigate north and expel a target south ($T = 152$ s, 170 s, 205 s); one USV navigates south and expels a target north ($T = 152$ s, 170 s). Finally, the last six USVs cooperatively hunt all the other targets at $T = 227$ s. It shows that DPOMH-PPO has strong adaptability and possesses the ability to continue to perform the task when half of the USVs are removed.

6. Conclusions

To deal with the problem of a USV fleet hunting multiple targets, based on the idea of USV's autonomous decision-making during operations, the DPOMH-PPO algorithm was developed. Through the comprehensive analysis of the encirclement and capture scenarios, the problem was modeled as Dec-POMDP, and then rational observation space, reward function, action space and network structure were designed. Finally, the simulation environment was built, and the USV fleet hunting strategy was trained with the centralized training and decentralized execution framework. The experimental results show that:

1) DPOMH-PPO can make the USVs in a fleet learn and cooperate in different test scenarios with different numbers of agents, and complete the multi-target hunting task quickly and effectively.
2) The ablation experiment proved that the feature embedded block involving CAP and CMP layers proposed in this work was the key to the performance improvement of the algorithms, and the contribution of the CAP layer was prominent.
3) Compared with the other algorithms, DPOMH-PPO shows obvious advantages in learning efficiency, generalization and robustness, and has the potential to promote practical applications.

A limitation of the current work is that we adopted a regularized escape strategy. This can be improved by implementing the target as an RL agent and training both sides simultaneously similar to Ref. [13]. In the future, scenarios near the coastline or islands, and heterogeneous USVs cooperative hunting control will be explored.

In addition, we aim to consider more realistic applications, and further explore the sim-to-real transfer and real-world applications to the multi-agent systems such as UAVs.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to acknowledge the financial support from National Natural Science Foundation of China(Grant No. 61601491), Natural Science Foundation of Hubei Province, China (Grant No. 2018CFC865) and Military Research Project of China(-Grant No. YJ2020B117).

References

[1] Darwish A, Hassanien AE, Das S. A survey of swarm and evolutionary computing approaches for deep learning[J]. Artif Intell Rev 2020;53(3):1767–812.
[2] Xu D, Chen G. The research on intelligent cooperative combat of UAV cluster with multi-agent reinforcement learning[J]. Aerospace Systems 2022;5.
[3] Fan J, Li D, Li R, et al. Analysis on MAV/UAV cooperative combat based on complex network[J]. Defence Technology 2020;16(1):150–7.
[4] Li S, Chen M, Wang Y, et al. Air combat decision-making of multiple UCAVs based on constraint strategy games[J]. Defence Technology 2022;18(3):368–83.
[5] Wang S, Zhang JQ, Yang SH, et al. Research on development status and combat applications of USVs in worldwide[J]. Fire Control Command Control 2019;44(2):11–5.
[6] Sun W, Tsiotras P, Lolla T, et al. Multiple–pursuer/one–evader pursuit–evasion game in dynamic flowfields[J]. J Guid Control Dynam 2017;40(7):1627–37.
[7] Muro C, Escobedo R, Spector L, et al. Wolf-pack (Canis lupus) hunting strategies emerge from simple rules in computational simulations[J]. Behav Process 2011;88(3):192–7.
[8] Janosov M, Virágh C, Vásárhelyi G, et al. Group chasing tactics: how to catch a faster prey[J]. New J Phys 2017;19(5):053003.
[9] Silver D, Huang A, Maddison CJ, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature 2016;529(7587):484–9.
[10] Ecoffet A, Huizinga J, Lehman J, et al. First return, then explore[J]. Nature 2021;590(7847):580–6.
[11] Vinyals O, Babuschkin I, Czarnecki WM, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning[J]. Nature 2019;575(7782):350–4.
[12] Silver D, Hubert T, Schrittwieser J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. Science 2018;362(6419):1140–4.
[13] Baker B, Kanitscheider I, Markov T, et al. Emergent tool use from multi-agent autocurricula[J]. 2019. arXiv preprint arXiv:1909.07528.
[14] Bai H, George J, Chakrabortty A. Hierarchical control of multi-agent systems using online reinforcement learning[C]. In: 2020 American control conference (ACC). IEEE; 2020. p. 340–5.
[15] Training AI to win a dogfight. https://www.darpa.mil/news-events, 2019-05-08.
[16] Fu X, Wang H, Xu Z. Research on cooperative pursuit strategy for multi-UAVs based on DE-MADDPG algorithm[J]. Acta Aeronautica Astronautica Sinica 2021;42:325311.
[17] Wang Y, Dong L, Sun C. Cooperative control for multi-player pursuit-evasion games with reinforcement learning[J]. Neurocomputing 2020:412.
[18] Wan K, Wu D, Zhai Y, et al. An improved approach towards multi–agent pursuit–evasion game decision–making using deep reinforcement learning [J]. Entropy 2021;23(11):1433.
[19] Souza C D, Newbury R, Cosgun A, et al. Decentralized multi-agent pursuit using deep reinforcement learning. IEEE Robotics and Automation Letters 2021;6(3):4552–9.
[20] Fujimoto S, Hoof H, Meger D. Addressing function approximation error in actor-critic methods[J]. 2018. arXiv preprint arXiv:1802.09477.
[21] Hüttenrauch M, Adrian S, Neumann G. Deep reinforcement learning for swarm systems[J]. J Mach Learn Res 2019;20(54):1–31.
[22] Ma J, Yao W, Dai W, et al. Cooperative hunting control for a group of targets by decentralized robots with collision avoidance[C]. In: 2018 37th Chinese control conference (CCC). IEEE; 2018. p. 6848–53.
[23] Yu C, Dong Y, Li Y, et al. Distributed multi-agent deep reinforcement learning for cooperative multi-robot pursuit[J]. J Eng 2020;2020(13):499–504.
[24] Zheng L, Yang J, Cai H, et al. MAgent: a many-agent reinforcement learning platform for artificial collective intelligence[C]. Proc AAAI Conf Artif Intell 2018;32(1).
[25] Oliehoek FA, Amato C. A concise introduction to decentralized POMDPs[M]. Springer; 2016.
[26] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. 2017. arXiv:1707.06347.
[27] Schulman J, Levine S, Abbeel P, et al. Trust region policy optimization[C]. In: International conference on machine learning. PMLR; 2015. p. 1889–97.
[28] Schulman J, Moritz P, Levine S, et al. High-dimensional continuous control using generalized advantage estimation[J]. 2015. arXiv preprint arXiv:1506.02438.
[29] Šošic A, KhudaBukhsh W, Zoubir A, et al. Inverse reinforcement learning in swarm systems.[C]. In: International conference on autonomous agents and multiagent systems; 2017. p. 1413–21.
[30] Lowe R, Wu YI, Tamar A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments[J]. Adv Neural Inf Process Syst 2017:30.
[31] Long Q, Zhou Z, Gupta A, et al. Evolutionary population curriculum for scaling

multi-agent reinforcement learning[J]. 2020. arXiv preprint arXiv:2003.10423.
[32] Xu G, Zhao Y, Liu H. Pursuit and evasion game between UVAs based on multi-agent reinforcement learning[C]. In: 2019 Chinese automation congress (CAC). IEEE; 2019. p. 1261–6.
[33] Yi G, Liu Z, Zhang JQ, et al. A USV heading tracking control method based on improved terminal sliding mode control[J]. Electron Opt Control 2020;27(10):12–6. 21.
[34] Hüttenrauch M, Šošić A, Neumann G. Local communication protocols for learning complex swarm behaviors with deep reinforcement learning[C]. In: International conference on swarm intelligence. Cham: Springer; 2018. p. 71–83.
[35] Gretton A, Borgwardt KM, Rasch MJ, et al. A kernel two-sample test[J]. J Mach Learn Res 2012;13(1):723–73.
[36] Foerster J, Farquhar G, Afouras T, et al. Counterfactual multi-agent policy gradients[C]. Proc AAAI Conf Artif Intell 2018;32(1).
[37] Rashid T, Samvelyan M, Schroeder C, et al. Qmix: monotonic value function factorisation for deep multi-agent reinforcement learning[C]. In: International conference on machine learning. PMLR; 2018. p. 4295–304.
[38] Yu C, Velu A, Vinitsky E, et al. The surprising effectiveness of MAPPO in cooperative, multi-agent games[J]. 2021. arXiv preprint arXiv:2103.01955.